

Lab16 – Config Data

In this exercise, you will first create a ConfigMap from a YAML configuration file as a source. Later, you'll create a Pod, consume the ConfigMap as Volume and inspect the key-value pairs as files. Also you will first create a Secret from literal values. Next, you'll create a Pod and consume the Secret as environment variables. Finally, you'll print out its values from within the container.

ConfigMap

1. Inspect the YAML configuration file named `application.yaml`

```
dev:
  url: http://dev.bar.com
  name: Developer Setup
prod:
  url: http://foo.bar.com
  name: My Cool App
```

2. Create a new ConfigMap named `app-config` from that file.

```
vagrant@kubernetes:~/lab05$ kubectl create configmap app-config --from-file=application.yaml
configmap/app-config created
vagrant@kubernetes:~/lab05$ kubectl get configmap app-config -o yaml
apiVersion: v1
data:
  application.yaml: |-
    dev:
      url: http://dev.bar.com
      name: Developer Setup
    prod:
      url: http://foo.bar.com
      name: My Cool App
kind: ConfigMap
metadata:
  creationTimestamp: "2023-07-20T13:34:34Z"
  name: app-config
  namespace: default
  resourceVersion: "38014"
  uid: 6122c55b-4738-46d8-bee6-41852433f7b1
vagrant@kubernetes:~/lab05$
```

3. Create a Pod named `backend` that consumes the ConfigMap as Volume at the mount path `/etc/config`. The container runs the image `nginx:1.23.4-alpine`.

```

vagrant@kubernetes-control-plane:~/lab05$ kubectl run backend --image=nginx:1.23.4-alpine -o yaml --dry-run=client --restart=Never > pod.yaml
vagrant@kubernetes-control-plane:~/lab05$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: backend
  name: backend
spec:
  containers:
    - image: nginx:1.23.4-alpine
      name: backend
      resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
  status: {}
vagrant@kubernetes-control-plane:~/lab05$

vagrant@kubernetes-control-plane:~/lab05$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab05$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: backend
  name: backend
spec:
  containers:
    - image: nginx:1.23.4-alpine
      name: backend
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-config
vagrant@kubernetes-control-plane:~/lab05$
vagrant@kubernetes-control-plane:~/lab05$ kubectl apply -f pod.yaml
pod/backend created
vagrant@kubernetes-control-plane:~/lab05$ _

```

4. Shell into the Pod and inspect the file at the mounted Volume path. You must find the file `application.yaml` with the expected YAML content.

```

vagrant@kubernetes-control-plane:~/lab05$ kubectl exec backend -it -- /bin/sh
/ # cd /etc/conf
/bin/sh: cd: can't cd to /etc/conf: No such file or directory
/ # cd /etc/config/
/etc/config # cat application.yaml
dev:
  url: http://dev.bar.com
  name: Developer Setup
prod:
  url: http://foo.bar.com
  name: My Cool App/etc/config #
/etc/config # exit
vagrant@kubernetes-control-plane:~/lab05$ _

```

Secret

5. Create a new Secret named `db-credentials` with the key/value pair `db-password=password`.

```

vagrant@kubernetes-control-plane:~/lab05$ kubectl create secret generic db-credentials --from-literal=db-password=password
secret/db-credentials created
vagrant@kubernetes-control-plane:~/lab05$ kubectl get secret
NAME                TYPE      DATA      AGE
db-credentials      Opaque    1          24s
vagrant@kubernetes-control-plane:~/lab05$
vagrant@kubernetes-control-plane:~/lab05$ kubectl get secret -o yaml
apiVersion: v1
items:
- apiVersion: v1
  data:
    db-password: cGFzc3dk
  kind: Secret
  metadata:
    creationTimestamp: "2023-07-20T13:53:05Z"
    name: db-credentials
    namespace: default
    resourceVersion: "39727"
    uid: a18d22c6-f082-4d3e-b0a6-36ac242eb089
  type: Opaque
kind: List
metadata:
  resourceVersion: ""
vagrant@kubernetes-control-plane:~/lab05$ _

```

6. Create a Pod named `backend` that uses the Secret as environment variable named `DB\_PASSWORD` and runs the container with the image `nginx:1.23.4-alpine`.

```
vagrant@kubernetes-control-plane:~/lab05$ kubectl run backend --image=nginx:1.23.4-alpine -o yaml --dry-run=client --restart=Never > pod.yaml
vagrant@kubernetes-control-plane:~/lab05$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: backend
  name: backend
spec:
  containers:
    - image: nginx:1.23.4-alpine
      name: backend
      resources: {}
      dnsPolicy: ClusterFirst
      restartPolicy: Never
  status: {}
vagrant@kubernetes-control-plane:~/lab05$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab05$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: backend
spec:
  containers:
    - image: nginx:1.23.4-alpine
      name: backend
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: password
vagrant@kubernetes-control-plane:~/lab05$ kubectl delete pod backend
pod "backend" deleted
vagrant@kubernetes-control-plane:~/lab05$ kubectl apply -f pod.yaml
pod/backend created
vagrant@kubernetes-control-plane:~/lab05$
```

7. Shell into the Pod and print out the created environment variables. You should find `DB\_PASSWORD` variable.

```
vagrant@kubernetes-control-plane:~/lab05$ kubectl exec -it backend -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=backend
NGINX_VERSION=1.23.4
PKG_RELEASE=1
NJS_VERSION=0.7.11
DB_PASSWORD=password
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
TERM=xterm
HOME=/root
vagrant@kubernetes-control-plane:~/lab05$
```